

Rapport de Projet : Sécurisation et Déploiement Automatisé d'une Architecture AWS 3-Tiers (Terraform)



Contexte : Ce projet démontre ma curiosité à initialiser, sécuriser et déployer une infrastructure Cloud complète sur Amazon Web Services (AWS) en utilisant l'approche "Infrastructure as Code" (IaC) avec Terraform. Le projet est divisé en deux phases : la configuration d'un socle de sécurité respectant les bonnes pratiques AWS, suivie du déploiement d'une architecture réseau hautement disponible, résiliente et automatisée.

Technologies utilisées :

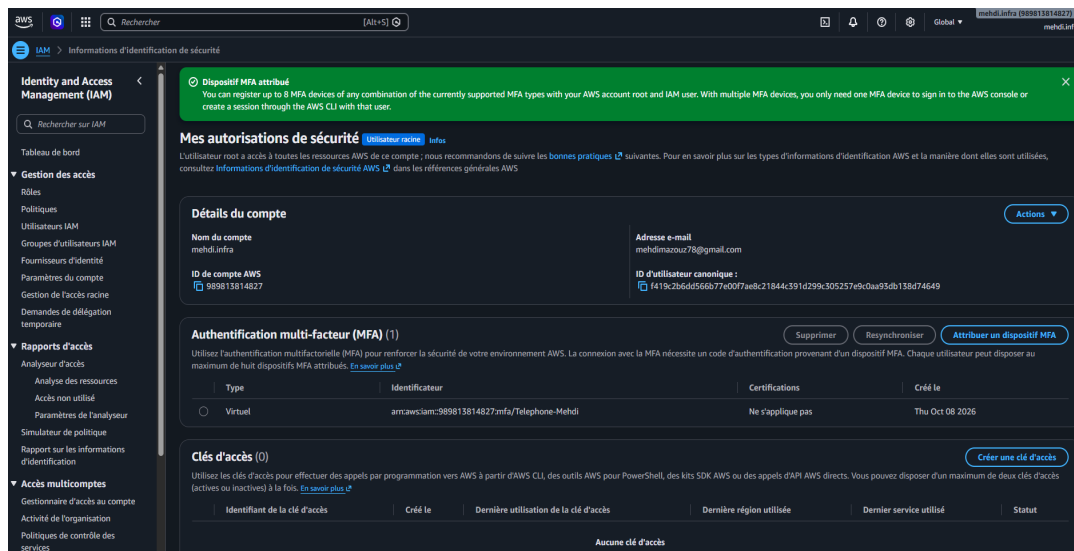
- **Fournisseur Cloud :** AWS (Région eu-west-3 - Paris)
- **Services AWS :** IAM, VPC, Subnets (Publics/Privés), Security Groups, Application Load Balancer (ALB), Auto Scaling Group (ASG), S3, DynamoDB.
- **Infrastructure as Code :** Terraform (CLI, HCL, Remote Backend).
- **Environnement de développement (Tooling) :** Visual Studio Code (VS Code), AWS CLI, Terminal.
- **Configuration Serveur :** Scripts Bash (user_data) pour l'installation d'Apache.

1. Sécurisation du Compte et Environnement de Travail (Tooling)

Avant de procéder au moindre déploiement, l'environnement de travail a été configuré pour répondre aux standards de sécurité de l'industrie.

1.1 Sécurisation des accès (IAM & MFA)

Le compte AWS Root a été immédiatement sécurisé via l'activation de l'authentification multifacteur (MFA). Conformément au principe de moindre privilège, les opérations courantes ont été déléguées à un utilisateur IAM dédié (admin-mehdi), empêchant ainsi l'utilisation du compte Root pour le déploiement.



1.2 Configuration de l'interface en ligne de commande (AWS CLI)

Des clés d'accès programmatiques (Access Key / Secret Key) ont été générées pour l'utilisateur IAM. L'environnement local a été configuré via `aws configure` pour pointer vers la région Paris (eu-west-3). La communication sécurisée entre le poste de travail et l'API AWS a été validée.

```
C:\Users\mehdi>aws configure
AWS Access Key ID [*****RGXT]:
AWS Secret Access Key [*****FKdQ]:
Default region name [eu-west-3 (la région de Paris)]: eu-west-3
Default output format [json]:

C:\Users\mehdi>aws sts get-caller-identity
{
  "UserId": "AIDA6M5MAGYV4J2PQOXS3",
  "Account": "989813814827",
  "Arn": "arn:aws:iam::989813814827:user/admin-mehdi"
}

C:\Users\mehdi>
```

1.3 Proof of Concept (PoC) : Premier déploiement IaC

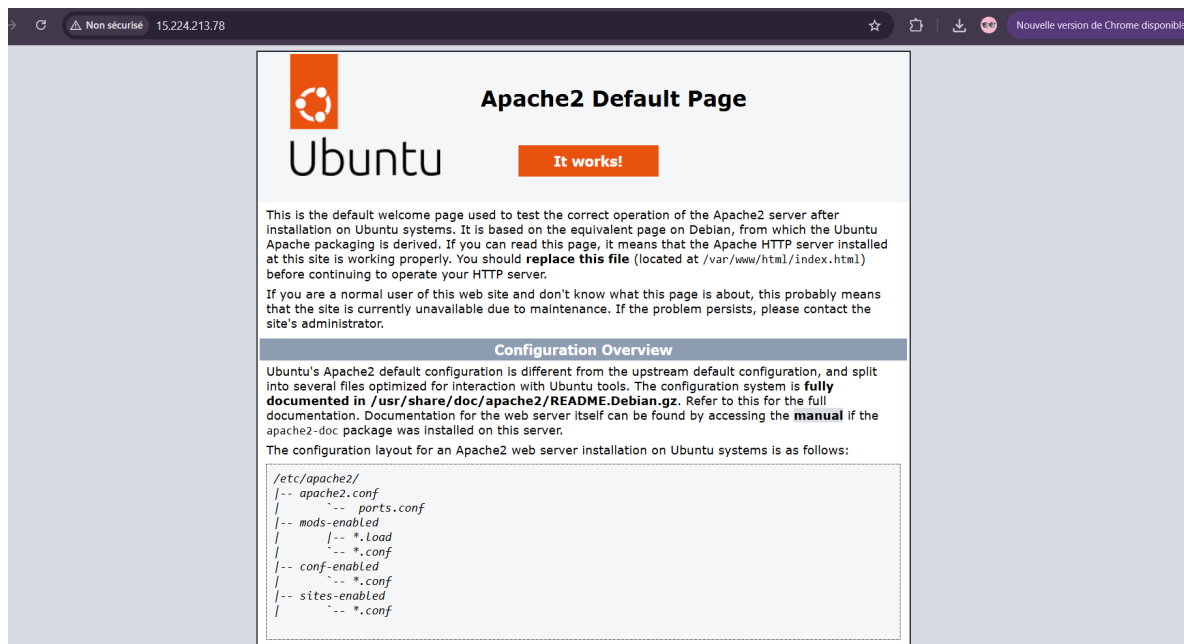
Pour valider le pipeline Terraform, un script HCL initial a été rédigé pour déployer une instance EC2 Ubuntu de base (t3.micro). Le succès du flux réseau a été vérifié par l'affichage d'un serveur web Apache rudimentaire sur l'IP publique générée par Terraform.

```
Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

ip_public = "15.224.213.78"
PS C:\Users\mehdi\aws-web-infra>
```



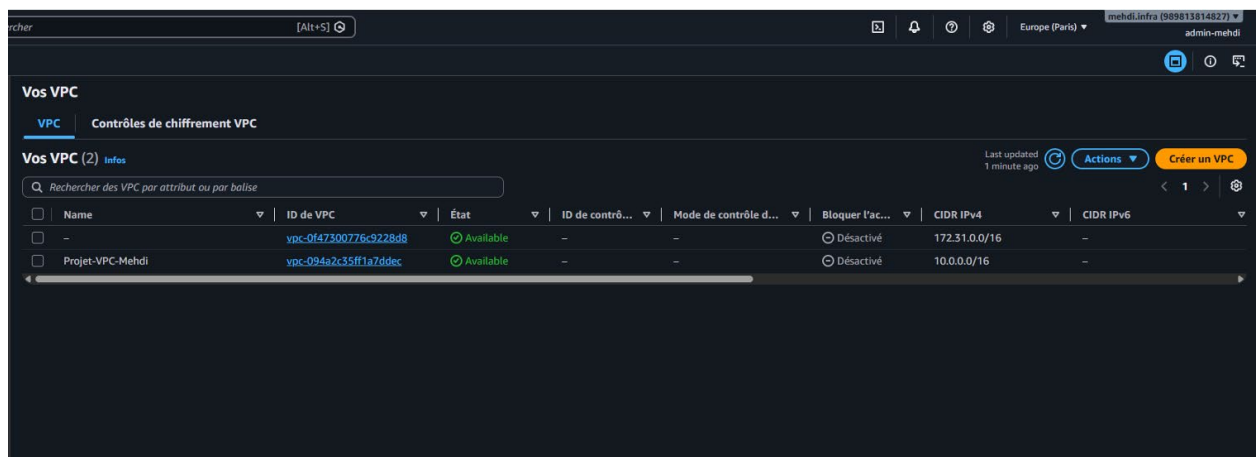
PARTIE 2 : Conception de l'Architecture 3-Tiers et Haute Disponibilité

Une fois le pipeline validé, le PoC a été détruit pour laisser place au déploiement d'une véritable architecture de production, pensée pour la tolérance aux pannes.

2.1 Réseau et Sécurité (Approche Zero Trust)

L'infrastructure ne repose pas sur le réseau par défaut d'AWS. Un réseau virtuel dédié (VPC) a été codé de zéro avec une segmentation stricte :

- Des sous-réseaux publics pour héberger le point d'entrée.
- Des sous-réseaux privés pour isoler les serveurs d'Internet.
- Des Groupes de Sécurité (Security Groups) configurés en cascade, où les instances web n'acceptent le trafic que s'il provient strictement du Load Balancer.



2.2 Travail collaboratif : Configuration du Remote State

Pour permettre à une équipe DevOps de collaborer sur cette infrastructure sans risque de conflit, le fichier d'état de Terraform (`terraform.tfstate`) a été externalisé.

- **Stockage** : Un bucket Amazon S3 chiffré sert de référentiel central.
- **Verrouillage (State Locking)** : Une table Amazon DynamoDB empêche les exécutions simultanées.

L'extrait de code HCL ci-dessous illustre la déclaration de ce backend distant, garantissant un environnement de déploiement synchronisé pour l'ensemble de l'équipe :

```
backend "s3" {
    bucket          = "projet-terraform-state-mehdi-123"
    key             = "projet-vpc/terraform.tfstate"
    region          = "eu-west-3"
    dynamodb_table = "terraform-state-lock-mehdi"
    encrypt         = true
}
```

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Warning: Deprecated Parameter

  on main.tf line 13, in terraform:
  13:   dynamodb_table = "terraform-state-lock-mehdi"

The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

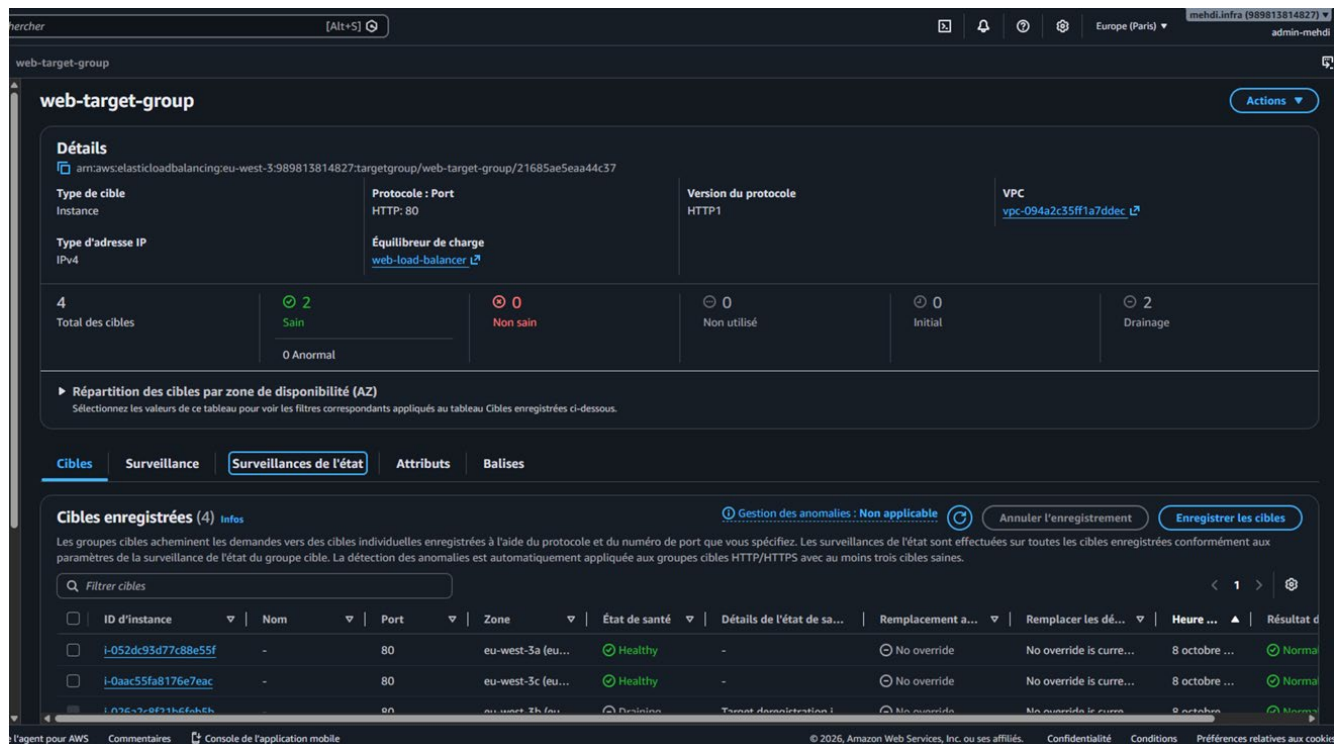
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\mehdi\aws-web-infra>
```

2.3 Haute Disponibilité et "Zero-Touch Provisioning"

Pour garantir la résilience du service, un Application Load Balancer (ALB) a été déployé sur plusieurs zones de disponibilité. Il est couplé à un Auto Scaling Group (ASG) configuré pour maintenir deux serveurs actifs en permanence. Les serveurs sont approvisionnés de manière 100% autonome : un script bash (user_data) s'exécute au lancement de l'instance pour installer et configurer Apache, supprimant tout besoin d'intervention manuelle en SSH.



PARTIE 3 : Gestion d'incidents (Troubleshooting) et FinOps

La mise en place de cette architecture a nécessité des compétences en diagnostic opérationnel :

- Résolution d'erreurs de routage** : Lors de la phase de test, les instances ont affiché un statut *Unhealthy*. L'analyse a permis d'identifier un décalage de zone de disponibilité entre l'ALB et l'ASG, corrigé via un *Rolling Update* avec Terraform.
- Gestion du cycle de vie (FinOps)** : En fin de projet, l'ensemble des ressources a été détruit de manière automatisée (terraform destroy). Un problème de désynchronisation de la table DynamoDB a été résolu en outrepassant temporairement le verrouillage (*-lock=false*), garantissant la suppression totale de l'environnement et une facturation maintenue à 0€.